

GEF Secure — Análisis Estratégico del Proyecto y Planificación del MVP

Documento generado mediante análisis profundo del código fuente, workflow n8n y documentación de proyecto. Las fuentes se identifican explícitamente en cada sección:

- **[PDF]** → información del documento de tesis
- **[Word]** → información del borrador de roadmap funcional
- **[Código]** → información extraída del análisis del código fuente real
- **[Análisis]** → conclusión o recomendación propia del análisis

Tabla de Contenidos

- [Estado Actual del Proyecto](#)
- [Comprensión del Producto](#)
- [Auditoría Técnica y Funcional](#)
- [Definición del MVP](#)
- [Roadmap por Iteraciones](#)
- [Product Backlog](#)
- [Mejoras Propuestas](#)
- [Arquitectura Objetivo](#)
- [Riesgos del Proyecto](#)
- [Próximos Pasos](#)
- [Conclusión](#)

1. Estado Actual del Proyecto

Resumen ejecutivo

[Código] El proyecto GEF Secure tiene una base de código funcional con las siguientes características:

Componente	Estado	Observaciones
Backend Spring Boot	Operativo	CRUD completo para Activos, Entornos, Vulnerabilidades, Usuarios
Frontend React	Operativo	Dashboard, Inventario, Kanban, Configuración implementados
Base de datos PostgreSQL	Operativo	7 tablas + 1 vista. Esquema inicial completo
n8n Workflow	Inactivo	"active": false — el scheduler nunca dispara
Trigger de escaneo	Roto	URL placeholder y payload incorrecto en Spring Boot
Autenticación	Ausente	No implementada (sin password_hash en tabla users)
Tests	Mínimos	Un solo test (contextLoads()) sin lógica de negocio
MTTR	Ausente	El campo resolvedAt existe pero no hay cálculo ni exposición

Stack tecnológico confirmado

[Código]

```
Backend:   Spring Boot 3 + Java 21 (Virtual Threads) + Gradle 8
Frontend:  React 18 + TypeScript + Vite + Tailwind CSS
Base de datos: PostgreSQL (volumen externo – requiere pre-creación manual)
Workflow:  n8n (fair-code) – escribe directamente a PostgreSQL
Proxy dev: Vite → localhost:8081
Proxy prod: nginx → backend:8080 (interno Docker)
Orquestación: Docker Compose multi-servicio
```

Servicios Docker Compose

[Código]

Servicio	Puerto	Estado
postgres	5432	Saludable (health check configurado)
n8n	5678	Inactivo (workflow no activado)
pgadmin	5050	Operativo
backend	8081	Operativo
frontend (nginx)	3000	Operativo

2. Comprensión del Producto

Problema que resuelve

[PDF] Las organizaciones gestionan sus vulnerabilidades de software de forma manual, reactiva y fragmentada. Los equipos de seguridad no tienen visibilidad centralizada del inventario de activos, no priorizan correctamente según el contexto del negocio (criticidad del entorno), y no pueden demostrar métricas de tiempo de respuesta (MTTR) ante auditorías.

[PDF] El gap no está en detectar vulnerabilidades (ya existen herramientas como Snyk, Dependabot), sino en **gestionarlas de forma contextualizada, trazable y auditable** dentro del ciclo de vida de una organización.

Solución propuesta

[PDF] GEF Secure es un sistema SaaS de gestión automatizada de vulnerabilidades que:

- Detecta vulnerabilidades cruzando el inventario interno de activos contra fuentes externas (GitHub Advisory Database, CISA KEV)
- Ajusta la prioridad de cada vulnerabilidad según la criticidad del entorno de negocio donde vive el activo
- Proporciona un pipeline de gestión (Detectada → En Análisis → Resuelta) con trazabilidad completa
- Calcula MTTR como KPI central de eficiencia del equipo de seguridad
- Notifica proactivamente al equipo via Slack con informes contextualizados

Visión del proyecto

[PDF] Convertirse en la plataforma de referencia para la gestión automatizada de vulnerabilidades en organizaciones medianas de habla hispana, diferenciándose por la integración de contexto de negocio en la priorización del riesgo.

Propuesta de valor diferencial

[PDF] A diferencia de herramientas que solo detectan, GEF Secure conecta la vulnerabilidad técnica con el impacto de negocio real: una vulnerabilidad ALTA en un activo de producción crítica se convierte en CRÍTICA. La misma vulnerabilidad en un entorno de desarrollo baja a MEDIA.

Usuarios objetivo

[PDF]

- **Usuario primario:** Analistas de seguridad y DevSecOps engineers en organizaciones medianas
- **Usuario secundario:** CTOs y responsables de cumplimiento que consumen los reportes
- **Contexto académico:** Tesis de UTN FRM — el sistema funciona también como prototipo demostrativo

Funcionalidades planificadas en el documento Word

[Word] El documento Word presenta las siguientes categorías de implementación:

Categoría	Funcionalidades
Gestión de Activos	CRUD, scan por activo, scan por entorno, scan global
Gestión de Vulnerabilidades	Detección automática, priorización contextual, ciclo de vida
Dashboard	Métricas en tiempo real, MTTR, distribuciones de severidad
Autenticación	JWT, roles (Admin/Auditor), control de acceso
Notificaciones	Slack por criticidad, resúmenes diarios
Deployment	Dockerización completa, variables de entorno
n8n	Automatización del pipeline, credenciales portables

Funcionalidades imprescindibles para el MVP

[Análisis]

- Inventario de activos con scan automático funcional
- Detección de vulnerabilidades vía n8n + GitHub Advisory + CISA KEV
- Priorización contextual por criticidad del entorno
- Gestión del ciclo de vida (Detectada → En Análisis → Resuelta)
- Dashboard con métricas básicas y MTTR
- Notificaciones Slack operativas

Funcionalidades secundarias (post-MVP)

[Análisis]

- Autenticación/autorización (el sistema actual usa usuarios sin contraseña — solo para asignación)
- Importación masiva de activos desde CSV
- API pública para integraciones externas
- Reportes exportables en PDF
- SLA tracking avanzado

Funcionalidades a eliminar del MVP

[Análisis]

- `WebhookController` en Spring Boot: los endpoints `/api/webhook/*` son código muerto (n8n escribe directo a PostgreSQL). Documentar como dead code — no eliminar aún, pero tampoco desarrollar.
- Gestión de listas de ignorados desde el frontend: el flujo n8n ya la maneja automáticamente.

3. Auditoría Técnica y Funcional

3.1 Auditoría del Producto

Coherencia

[Análisis] El producto tiene coherencia conceptual sólida. La propuesta de valor (vulnerabilidad + contexto de negocio = prioridad real) está efectivamente implementada en el motor de riesgo del n8n. Sin embargo, existe una brecha entre la visión del PDF y la implementación actual:

- El PDF menciona MTTR como KPI central → **no está implementado en ninguna capa**
- El Word sugiere autenticación JWT → **no hay `password_hash` en la tabla `users`**
- El Word menciona "scan inteligente por activo" → **el trigger envía payload incorrecto y n8n está inactivo**

Fortalezas

- Motor de riesgo contextualizado (CVSS + criticidad de entorno + CISA KEV + zero-day) — genuinamente diferenciador
- Arquitectura de notificaciones por tipo de scan (global / entorno / activo) — bien pensada
- Stack moderno y apropiado para el MVP
- Pipeline de errores en n8n con persistencia automática en `system_errors`
- Health checks de Docker Compose ya implementados

Debilidades

- El feature más importante del producto (el scan automático) está roto en 3 puntos simultáneamente
- Cero tests de negocio — cualquier refactor rompe silenciosamente
- MTTR ausente siendo el KPI central del PDF
- `CVSS` almacenado como `VARCHAR` — impide ordenamiento y filtros numéricos correctos

Diferenciadores en riesgo

[Análisis] Si el scan no funciona, el diferenciador principal desaparece. Toda la propuesta de valor depende de que el pipeline n8n → PostgreSQL ejecute correctamente.

3.2 Auditoría de Arquitectura

Diagrama de arquitectura actual

```
graph TD
    FE[Frontend React\nnginx :3000]
    BE[Backend Spring Boot\n:8081]
    N8N[n8n Workflow\n:5678]
    DB[(PostgreSQL\n:5432)]
    GHSA[GitHub Advisory API]
    CISA[CISA KEV API]
    SL[Slack]

    FE -->|/api/*| BE
```

```

BE -->|JDBC| DB
BE -->|POST webhook| N8N
N8N -->|SQL directo| DB
N8N -->|HTTP GET| GHSA
N8N -->|HTTP GET| CISA
N8N -->|API| SL

style N8N fill:#e8a838,color:#000
style DB fill:#336791,color:#fff
style BE fill:#6db33f,color:#fff
style FE fill:#61dafb,color:#000

```

Decisión arquitectónica fijada por el usuario

[Decisión del usuario — No modificar] *n8n* escribe directamente a PostgreSQL, sin pasar por la API de Spring Boot. Los endpoints `WebhookController` (`/api/webhook/*`) son código muerto intencional. Esta decisión se mantiene como está.

Escalabilidad

[Análisis] La arquitectura actual es adecuada para MVP y organizaciones pequeñas-medianas:

- Java 21 Virtual Threads habilita concurrencia alta sin overhead de hilos del SO
- *n8n* puede escalarse horizontalmente en etapas posteriores
- PostgreSQL aguanta el volumen esperado de vulnerabilidades sin necesidad de optimización inmediata

Riesgo de escala: Si se gestionan >100 activos con escaneos diarios, la tabla `vulnerabilities_audit` puede crecer exponencialmente sin el `ON CONFLICT` faltante.

Mantenibilidad

[Análisis] Problemas detectados:

- Lógica de negocio crítica (matriz de riesgo) embebida en un Code node de *n8n* — no versionable, no testeable
- CVSS como `VARCHAR` obliga a conversiones en cada capa
- `AssetController` inyecta directamente `AssetRepository` saltando la capa de servicio

Deuda técnica potencial

Ítem	Impacto	Urgencia
CVSS como VARCHAR	Filtros incorrectos, bugs de ordenamiento	Media
Lógica de riesgo en Code node <i>n8n</i>	No testeable, no versionable	Alta (post-MVP)
<code>AssetController</code> bypass de servicio	Viola arquitectura en capas	Media
Tests inexistentes	Refactors ciegos	Alta
<code>N8N_ENCRYPTION_KEY</code> hardcodeada	Seguridad en producción	Alta

3.3 Auditoría de la Base de Datos

[Código] Tablas confirmadas en `init/01-create-tables.sql` :

Tabla	Propósito	Observaciones
environments	Entornos de negocio	OK
assets	Inventario de activos	cvss ausente (está en vulns)
vulnerabilities_audit	Registro central de vulns	cvss VARCHAR, no NUMERIC
vulnerabilities_ignored	Vulns sin activo interno	Sin columna reason ni asset_id
users	Asignación en Kanban	Sin password_hash — no es auth
system_errors	Errores del pipeline n8n	OK
scan_reports	Historial de escaneos	Referenciada en n8n, verificar existencia

[Análisis] La tabla `vulnerabilities_ignored` no tiene columnas `reason` ni `asset_id` que el documento Word asume. Esto fue corregido en el análisis.

3.4 Auditoría del Flujo n8n

[Código] Bugs críticos identificados (ver sección 7 para detalle completo):

ID	Nodo afectado	Severidad
BUG-N8N-07	Edit Fields	CRÍTICO — filtros reciben "undefined", 0 vulns detectadas vía webhook
BUG-N8N-10	Workflow raíz	CRÍTICO — "active": false, scheduler nunca dispara
BUG-N8N-04	Persist_Audit_Findings	ALTO — sin ON CONFLICT, duplicados ilimitados
BUG-N8N-02	Fetch_GitHub_Advisories	ALTO — sin parámetros, solo 30 advisories por ejecución
BUG-N8N-05/06	Check_Internal_Assets, Log_Ignored	ALTO — SQL injection via interpolación
BUG-N8N-08	Fetch_CISA_KEV	MEDIO — sin caché, descarga completa diaria
BUG-N8N-01	Fetch_CISA_KEV → GitHub	MEDIO — ejecución serie innecesaria
BUG-N8N-09	Notify_* (3 nodos Slack)	MEDIO — URL hardcodeda a localhost:3000

3.5 Auditoría del Frontend

[Código] Bugs detectados:

Componente	Bug	Impacto
Kanban.tsx	filterDate defaults a hoy → tablero vacío para nuevos usuarios	UX crítica
types/index.ts	VulnPriority = 'CRITICA' 'ALTA'... pero datos reales son inglés (CRITICAL HIGH)	Posibles errores de tipado

Inventory.tsx	handleScan() muestra toast de éxito independientemente del resultado n8n	Feedback falso al usuario
---------------	--	---------------------------

[Análisis] La página Settings NO está vacía — tiene CRUD completo de usuarios y log de errores del sistema. Esta funcionalidad está completamente implementada.

3.6 Auditoría del Backend

[Código] Bugs detectados:

Componente	Bug	Impacto
N8nWebhookService	Envía {assetId, software, version, ecosystem} en lugar de {activo_name}	Scan siempre global, nunca por activo
N8nWebhookService	Swallows exceptions silenciosamente	Errores ocultos al frontend
AssetController	Carga el activo dos veces (duplicado de query)	Performance y arquitectura
AssetController	Retorna 200 aunque n8n falle	Frontend no puede detectar el fallo
DashboardService	No calcula MTTR en ningún punto	KPI central ausente
VulnerabilityAuditService	No valida transiciones de estado	Cualquier transición es aceptada
application.properties	URL fallback tiene path incorrecto (gef-scan vs auditoria-automatizada)	Falla sin Docker

4. Definición del MVP

Principios de definición

[Análisis] El MVP debe:

- Demostrar que el pipeline completo funciona end-to-end (inventario → scan → vulnerabilidades → gestión)
- Ser demostrable ante el tribunal de tesis
- Validar la hipótesis diferenciadora: priorización contextual por criticidad del entorno
- Ser desplegable con `docker compose up`

Funcionalidades del MVP

#	Nombre	Descripción	Problema que resuelve	Prioridad	Complejidad	Dependenc
MVP-01	Pipeline de scan funcional	Fix completo del trigger: URL, payload, activación del workflow	Sin esto el producto no existe	CRÍTICA	Media	—

MVP-02	Gestión de inventario	CRUD de activos y entornos con criticidad de negocio	Permite configurar qué se monitorea	CRÍTICA	Baja	Implementa
MVP-03	Detección automática de vulnerabilidades	n8n consultando GitHub Advisory + CISA KEV diariamente	Core del producto	CRÍTICA	Media	MVP-01
MVP-04	Priorización contextual	Motor de riesgo ajustando prioridad por criticidad de entorno	Diferenciador principal	CRÍTICA	Baja	MVP-03
MVP-05	Ciclo de vida de vulnerabilidades	Transiciones Detectada → En Análisis → Resuelta con validación	Gestión trazable	ALTA	Baja	MVP-03
MVP-06	Dashboard con MTTR	Métricas centrales incluyendo Mean Time To Remediate	KPI del PDF	ALTA	Media	MVP-05
MVP-07	Notificaciones Slack	Alertas por tipo de scan con URL configurable	Visibilidad del equipo	ALTA	Baja	MVP-01
MVP-08	Credenciales n8n portables	Setup automático de credenciales desde variables de entorno	Deployability	ALTA	Media	—
MVP-09	Deployment reproducible	.env.example, pgdata no-externo, documentación de setup	Demostración ante tribunal	ALTA	Baja	—
MVP-10	Kanban de gestión	Tablero para asignar y actualizar vulnerabilidades	Flujo de trabajo	MEDIA	Baja	Implementa

Funcionalidades fuera del MVP

[Análisis]

- Autenticación JWT — los usuarios existen pero sin contraseña. Implementar en v2.
- Importación CSV de activos — no aporta valor demostrativo
- Reportes PDF exportables — complejidad vs. valor bajo en MVP
- API pública — post-MVP cuando haya usuarios reales

5. Roadmap por Iteraciones

```
gantt
    title Roadmap GEF Secure – MVP
    dateFormat YYYY-MM-DD
    section Iteración 0 (Infraestructura)
    Fix deployment & Docker      :crit, i0a, 2026-08-01, 3d
    .env.example & secrets      :i0b, after i0a, 2d
    section Iteración 1 (Pipeline)
    Fix webhook URL & payload   :crit, i1a, after i0b, 2d
    Activar workflow n8n        :crit, i1b, after i0b, 2d
    Credenciales portables      :i1c, after i1b, 3d
    section Iteración 2 (Correcciones Críticas)
    ON CONFLICT vulnerabilities :i2a, after i1c, 1d
    SQL injection fixes n8n     :i2b, after i1c, 2d
    GitHub API params & paginación :i2c, after i1c, 2d
    Error propagation frontend  :i2d, after i2c, 1d
    section Iteración 3 (Funcionalidades Core)
    MTTR en dashboard           :i3a, after i2d, 3d
    Validación transiciones      :i3b, after i2d, 2d
    Fix Kanban filterDate        :i3c, after i2d, 1d
    Fix VulnPriority tipos TS     :i3d, after i2d, 1d
    section Iteración 4 (Eficiencia n8n)
    CISA KEV caché              :i4a, after i3a, 2d
    Paralelizar fetches          :i4b, after i3a, 1d
    Retry en nodos HTTP          :i4c, after i3a, 1d
    section Iteración 5 (Deployment)
    Documentación deployment     :i5a, after i4a, 2d
    Smoke tests end-to-end       :i5b, after i5a, 2d
```

Iteración 0 — Infraestructura base

Objetivo: Que `docker compose up` funcione en cualquier máquina sin configuración manual.

Funcionalidad	Descripción	Criterio de aceptación
Eliminar pgdata: external: true	Cambiar a volumen gestionado por Compose	<code>docker compose up</code> crea el volumen automáticamente
Crear <code>.env.example</code>	Documentar todas las variables requeridas	El archivo existe con valores de ejemplo y comentarios

Mover secretos a .env	N8N_ENCRYPTION_KEY, credenciales pgAdmin, URL webhook	docker-compose.yml no contiene secretos hardcodeados
-----------------------	---	--

Resultado esperado: Primera instalación funciona en 5 minutos sin documentación adicional.

Riesgos: Pérdida de datos si se elimina volumen existente — documentar migración.

Iteración 1 — Pipeline de scan funcional

Objetivo: El botón "Disparar Scan" en el frontend produce vulnerabilidades reales en la base de datos.

Funcionalidad	Descripción	Criterio de aceptación
Fix URL webhook en docker-compose.yml	N8N_WEBHOOK_URL: http://n8n:5678/webhook/auditoria-automatizada	El nodo en n8n recibe el request
Fix payload N8nWebhookService	Enviar {activo_name: asset.getName()} para scan por activo	n8n recibe el nombre correcto del activo
Activar workflow n8n via API	Script setup-n8n.sh que activa el workflow en primer arranque	El scheduler corre a las 09:00
Habilitar n8n Public API	N8N_PUBLIC_API_ENABLED=true + N8N_API_KEY en docker-compose	La API de n8n responde en /api/v1/workflows
Fix Edit Fields en n8n	Agregar `	

Resultado esperado: Scan manual desde el frontend detecta vulnerabilidades reales del activo seleccionado.

Riesgos: n8n puede tardar en arrancar — setup-n8n.sh debe incluir retry loop.

Iteración 2 — Correcciones críticas de datos y seguridad

Objetivo: El pipeline produce datos correctos, sin duplicados y sin riesgos de inyección SQL.

Funcionalidad	Descripción	Criterio de aceptación
ON CONFLICT en Persist_Audit_Findings	ON CONFLICT (asset_id, cve_id, ghsa_id) WHERE status = 'DETECTADA' DO NOTHING	Dos scans del mismo activo no duplican vulnerabilidades
Parámetros GitHub Advisory API	Agregar type=reviewed, per_page=100 al nodo principal	Se procesan 100 advisories por ejecución (no 30)
Fix SQL injection en n8n	Migrar queries de executeQuery a modo GUI con binding	Package names con comillas no rompen el pipeline
Propagación de errores al frontend	N8nWebhookService relanza excepciones; AssetController retorna 500 si falla	Frontend muestra toast de error real cuando n8n falla
Consolidar AssetController	Eliminar segundo assetRepository.findById — usar data del servicio	Un solo query por operación de scan

Resultado esperado: Los datos en la DB son correctos, únicos y el frontend refleja el estado real del pipeline.

Riesgos: La migración de `executeQuery` a modo GUI puede alterar la lógica de filtros existente — validar con datos reales.

Iteración 3 — Funcionalidades core del producto

Objetivo: El producto cumple la propuesta de valor completa del PDF.

Funcionalidad	Descripción	Criterio de aceptación
MTTR en Dashboard	Query de avg(resolvedAt - detectedAt) para vulns RESUELTA; campo en DTO y card en frontend	El Dashboard muestra MTTR en días/horas
Validación de transiciones de estado	<code>InvalidStateTransitionException</code> + handler HTTP 409; solo transiciones válidas aceptadas	RESUELTA → DETECTADA retorna 409, no 200
Fix Kanban <code>filterDate</code>	Cambiar default de hoy a vacío (sin filtro de fecha)	Al abrir Kanban por primera vez se ven todas las vulns activas
Fix <code>VulnPriority</code> TypeScript	Cambiar tipo a <code>'CRITICAL' 'HIGH' 'MEDIUM' 'LOW'</code>	No hay discrepancias de tipado entre frontend y datos reales

Resultado esperado: El producto demuestra el ciclo completo de gestión de vulnerabilidades con métricas reales.

Riesgos: Cambio de tipo `VulnPriority` puede requerir actualizar estilos y renders que usan los valores en español.

Iteración 4 — Eficiencia del workflow n8n

Objetivo: El pipeline es confiable, eficiente y no produce falsos positivos por repetición.

Funcionalidad	Descripción	Criterio de aceptación
Caché CISA KEV en PostgreSQL	Tabla <code>cisa_kev_cache</code> ; refresh solo si >24h	Segundo scan del día no hace HTTP a CISA
Paralelizar CISA + GitHub Advisories	Ambos nodos arrancan desde <code>Edit Fields</code> en paralelo con Merge de espera	Tiempo total de pipeline -3 a -8 segundos
Retry en nodos HTTP	Retry on Fail: 3 intentos, 1s de espera en <code>Fetch_CISA_KEV</code> y <code>Fetch_GitHub_Advisories</code>	Timeout puntual no falla todo el workflow
Injector lanza scan automático	Después de <code>Upsert_Test_Asset</code> , POST al propio webhook con <code>{activo_name}</code>	Activo inyectado queda auditado en el mismo flujo
Fix <code>Random_Asset_Selector</code>	Cambiar <code>environment_id</code> random a id fijo del entorno DESARROLLO	Activos de prueba solo se crean en entorno de desarrollo

Resultado esperado: El pipeline es robusto ante fallos externos y eficiente en el uso de red.

Riesgos: La paralelización requiere validar que `Risk_Assessment_Engine` recibe correctamente el resultado de CISA (referencia directa al nodo, no al flujo de datos).

Iteración 5 — Deployment y cierre del MVP

Objetivo: El sistema es desplegable por un tercero sin asistencia.

Funcionalidad	Descripción	Criterio de aceptación
README.md de deployment	Pasos: prerequisites, clonar, configurar .env, docker compose up, verificar	Un desarrollador nuevo levanta el sistema en <15 minutos
docker-compose.prod.yml	Configuración de producción con variables externas	Separación clara entre dev y prod
Smoke tests end-to-end	Script que verifica: API responde, n8n activo, scan dispara, vuln aparece en DB	./smoke-test.sh pasa sin errores

Resultado esperado: El MVP está listo para demostración ante el tribunal.

6. Product Backlog

ID	Nombre	Descripción	Prioridad	Valor de negocio	Comp
PB-047	Fix URL webhook docker-compose	Reemplazar webhook/tu-id por webhook/auditoria-automatizada	CRÍTICA	Alto	Mínim
PB-048	Script setup-n8n.sh	Activar workflow y crear credenciales automáticamente al iniciar	CRÍTICA	Alto	Medio
PB-049	Habilitar n8n Public API	N8N_PUBLIC_API_ENABLED=true + N8N_API_KEY en docker-compose	CRÍTICA	Alto	Mínim
PB-056	Fix payload N8nWebhookService	Enviar {activo_name} en lugar de {assetId, software, version, ecosystem}	CRÍTICA	Alto	Baja
PB-N8N-07	Fix filtros Edit Fields n8n	Agregar 'todos' fallback cuando el body no incluye los campos	CRÍTICA	Alto	Mínim
PB-N8N-10	Activar workflow n8n	"active": true O via API en setup script	CRÍTICA	Alto	Mínim
PB-057	Fix pgdata external volume	Cambiar a volumen gestionado por Compose	ALTA	Alto	Mínim
PB-050	Crear .env.example	Documentar todas las variables de entorno requeridas	ALTA	Alto	Mínim

PB-051	Mover N8N_ENCRYPTION_KEY a .env	Eliminar secreto hardcodeado del docker-compose.yml	ALTA	Alto	Mínim
PB-N8N-04	ON CONFLICT en Persist_Audit_Findings	Evitar duplicados de vulnerabilidades en scans consecutivos	ALTA	Alto	Baja
PB-N8N-02	Parámetros GitHub Advisory API	Agregar type=reviewed, per_page=100 al nodo principal	ALTA	Alto	Mínim
PB-N8N-05	Fix SQL injection Check_Internal_Assets	Migrar a modo GUI de nodo Postgres con binding	ALTA	Alto	Medi
PB-N8N-06	Fix SQL injection Log_Ignored_Vulnerabilities	Misma corrección que PB-N8N-05	ALTA	Alto	Medi
PB-064	Propagar errores n8n al frontend	N8nWebhookService relanza excepciones; AssetController retorna 500	ALTA	Alto	Baja
PB-065	Mover webhook call a AssetService	Eliminar assetRepository de AssetController	ALTA	Medio	Baja
PB-002	Implementar MTTR	Query + campo en DTO + card en Dashboard	ALTA	Muy Alto	Medi
PB-003	Validación de transiciones de estado	InvalidStateException + handler 409	ALTA	Alto	Baja
PB-063	Fix Kanban filterDate default	Cambiar default de hoy a sin filtro	ALTA	Alto	Mínim
PB-059	Fix VulnPriority tipos TypeScript	Cambiar a 'CRITICAL' 'HIGH' 'MEDIUM' 'LOW'	ALTA	Medio	Mínim
PB-066	InvalidStateException	Crear excepción + handler en GlobalExceptionHandler (HTTP 409)	ALTA	Medio	Baja
PB-N8N-08	Caché CISA KEV en PostgreSQL	Tabla cisa_kev_cache con TTL de 24h	MEDIA	Alto	Medi
PB-N8N-01	Paralelizar CISA + GitHub	Ejecutar ambos fetches en paralelo	MEDIA	Medio	Baja
PB-N8N-09	Fix URL Slack hardcodeada	Reemplazar localhost:3000 por variable de entorno	MEDIA	Medio	Mínim

PB-N8N-03	Paginación GitHub Advisory API	Iterar páginas hasta agotar resultados	MEDIA	Alto	Alta
PB-060	Retry en nodos HTTP n8n	Retry on Fail: 3 intentos en CISA y GitHub	MEDIA	Alto	Mínim
PB-061	Endpoint scan por entorno	POST /api/environments/{id}/scan	MEDIA	Medio	Baja
PB-062	Endpoint scan global	POST /api/scan	MEDIA	Medio	Baja
PB-004	Paginación en VulnerabilityAuditController	Parámetros page y size en el endpoint de listado	MEDIA	Medio	Baja
PB-005	Campo resolvedAt en DTO	Exponer resolvedAt en la respuesta del frontend	MEDIA	Medio	Mínim
PB-N8N-11	Fix SQL injection Persist_Scan_Statistics	Sanitizar JSON.stringify antes de interpolar	MEDIA	Medio	Baja
PB-N8N-06B	Fix Inyector — environment_id fijo	Cambiar random a entorno DESARROLLO (id=1)	MEDIA	Medio	Mínim
PB-N8N-06C	Inyector lanza scan automático	POST al webhook después de Upsert_Test_Asset	MEDIA	Medio	Baja
PB-006	CVSS como NUMERIC en DB	Migración para cambiar tipo de columna	BAJA	Medio	Medi
PB-007	Tests de negocio (Spring Boot)	Tests de servicio para VulnerabilityAuditService, AssetService	BAJA	Alto	Alta
PB-008	Smoke tests end-to-end	Script smoke-test.sh que verifica el pipeline completo	BAJA	Alto	Medi
PB-009	README deployment	Documentación paso a paso	BAJA	Alto	Baja
PB-043	Docker health checks	Ya implementado — depends_on: condition: service_healthy	—	—	—

7. Mejoras Propuestas

Imprescindibles

MEJ-01 — Caché de CISA KEV en PostgreSQL

Problema que resuelve: CISA KEV se descarga completo (~1500 entradas, ~2-3 MB) en cada ejecución diaria, aunque CISA actualiza el catálogo semanalmente.

Beneficio: Elimina 6 de cada 7 llamadas HTTP a CISA por semana. Reduce el tiempo de ejecución del pipeline.

Implementación:

```
CREATE TABLE cisa_kev_cache (  
  id SERIAL PRIMARY KEY,  
  fetched_at TIMESTAMP NOT NULL DEFAULT NOW(),  
  data JSONB NOT NULL  
);
```

Nodo de decisión al inicio del flujo: si `NOW() - fetched_at < INTERVAL '24 hours'` → leer de DB; si no → fetch HTTP y actualizar.

Complejidad: Media | **Impacto:** Alto | **Prioridad:** ALTA

MEJ-02 — Propagación correcta de errores de n8n al usuario

Problema que resuelve: El usuario hace click en "Disparar Scan", ve un toast verde de éxito, pero n8n puede haber fallado silenciosamente. No hay forma de saber si el scan realmente ejecutó.

Beneficio: Feedback honesto al usuario. Permite diagnóstico inmediato de problemas.

Implementación: `N8nWebhookService` relanza la excepción (no la swallow). `AssetController` retorna 500 cuando el webhook falla. Frontend muestra toast de error con mensaje específico.

Complejidad: Baja | **Impacto:** Alto | **Prioridad:** CRÍTICA

MEJ-03 — MTTR (Mean Time To Remediate) en Dashboard

Problema que resuelve: El PDF define MTTR como KPI central. No está implementado en ninguna capa del sistema.

Beneficio: Permite medir la eficiencia real del equipo de seguridad. Es el diferenciador del producto ante herramientas de detección pura.

Implementación:

```
SELECT AVG(EXTRACT(EPOCH FROM (resolved_at - detected_at)) / 3600)  
FROM vulnerabilities_audit  
WHERE status = 'RESUELTA'  
  AND resolved_at IS NOT NULL  
  AND detected_at >= NOW() - INTERVAL '30 days';
```

Agregar `avgMtttrHours` al `DashboardStatsDTO` y al `DashboardService`. Card en el dashboard con valor en horas/días.

Complejidad: Media | **Impacto:** Muy Alto | **Prioridad:** ALTA

Muy recomendables

MEJ-04 — Paginación de GitHub Advisory API

Problema que resuelve: Sin paginación, el flujo procesa solo la primera página (30-100 advisories) de los >10.000 existentes. El sistema tiene cobertura de detección inferior al 1%.

Implementación: Loop en n8n que itera páginas usando el header `Link: <...>; rel="next"` . Condición de salida: no existe `rel="next"` o se alcanzó un máximo configurable de páginas.

Complejidad: Alta | **Impacto:** Muy Alto | **Prioridad:** MEDIA

MEJ-05 — Credenciales n8n portables via API

Problema que resuelve: Al deploy en un nuevo servidor, las credenciales de GitHub, PostgreSQL y Slack deben reconfigurarse manualmente en la UI de n8n. No es reproducible.

Implementación:

```
# setup-n8n.sh
curl -X POST http://n8n:5678/api/v1/credentials \
  -H "X-N8N-API-KEY: $N8N_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"name":"GitHub Token","type":"httpHeaderAuth","data":
{"name":"Authorization","value":"Bearer $GITHUB_TOKEN"}}'
```

El script crea las 3 credenciales (GitHub, PostgreSQL, Slack) y luego las mapea al workflow antes de activarlo.

Complejidad: Media | **Impacto:** Alto | **Prioridad:** ALTA

MEJ-06 — Extraer Risk Assessment Engine a código versionado

Problema que resuelve: La lógica de negocio más crítica del sistema (~100 líneas de JS que determina la prioridad final de cada vulnerabilidad) está embebida en un Code node de n8n. No es testeable, no tiene control de versiones legible, y cualquier cambio requiere editar JSON manualmente.

Implementación: Crear endpoint interno en Spring Boot `POST /api/internal/risk-assessment` que reciba los datos de la vulnerabilidad y retorne la prioridad calculada. El nodo n8n llama a este endpoint. La lógica queda en Java, con tests unitarios.

Complejidad: Alta | **Impacto:** Muy Alto (mantenibilidad) | **Prioridad:** MEDIA (post-MVP)

Opcionales

MEJ-07 — Autenticación JWT

Situación actual: La tabla `users` no tiene `password_hash` . Los usuarios sirven únicamente para asignación en el Kanban.

Beneficio: Seguridad real multi-usuario con roles (ADMIN/AUDITOR).

Complejidad: Alta | **Impacto:** Alto | **Prioridad:** BAJA (v2)

MEJ-08 — Filtros de ecosistema en scan por activo

Beneficio: Al escanear un activo npm específico, filtrar la GitHub Advisory API por `ecosystem=npm` reduce en un 70-80% el volumen de datos a procesar.

Complejidad: Baja | **Impacto:** Medio | **Prioridad:** BAJA

MEJ-09 — Reintentos en nodos HTTP de n8n

Beneficio: Timeouts puntuales de CISA o GitHub no fallan todo el workflow. El pipeline es más resiliente a fallas externas transitorias.

Implementación: Retry on Fail: 3 intentos, 1 segundo de espera en nodos Fetch_CISA_KEV y Fetch_GitHub_Advisories .

Complejidad: Mínima | **Impacto:** Alto | **Prioridad:** MEDIA

8. Arquitectura Objetivo

Visión de arquitectura del MVP

```
graph TB
    subgraph "Usuario"
        Browser[Navegador Web]
    end

    subgraph "Docker Compose"
        subgraph "Frontend"
            Nginx["nginx :3000\nSPA + Proxy /api/"]
        end

        subgraph "Backend"
            API["Spring Boot\nJava 21 VT\n:8081"]
            subgraph "Capas"
                Controllers[Controllers]
                Services[Services]
                Repositories[Repositories JPA]
            end
        end
    end

    subgraph "Automatización"
        N8N["n8n Workflow\n:5678"]
        SetupScript["setup-n8n.sh\nScript de inicialización"]
    end

    subgraph "Datos"
        PG["PostgreSQL\n:5432"]
    end

    subgraph "Fuentes Externas"
        GHSA[GitHub Advisory API]
        CISA[CISA KEV API]
        Slack[Slack API]
    end

    Browser --> Nginx
    Nginx --> |"/api/"| API
```

```
API --> Controllers --> Services --> Repositories --> PG
API -->|POST webhook trigger| N8N
N8N -->|SQL directo| PG
N8N -->|HTTP GET| GHSA
N8N -->|HTTP GET| CISA
N8N -->|Notificaciones| Slack
SetupScript -->|API v1| N8N

style N8N fill:#e8a838,color:#000
style PG fill:#336791,color:#fff
style API fill:#6db33f,color:#fff
style Nginx fill:#009639,color:#fff
```

Módulos y responsabilidades

Backend (Spring Boot)

Módulo	Responsabilidad
EnvironmentController	CRUD de entornos de negocio con criticidad
AssetController	CRUD de activos + trigger de scan
VulnerabilityAuditController	Listado, detalle, actualización de estado
DashboardController	Métricas agregadas incluyendo MTTR
UserController	CRUD de usuarios para asignación
SystemErrorController	Visualización de errores del pipeline n8n
N8nWebhookService	Comunicación con n8n — trigger de scans
GlobalExceptionHandler	Manejo centralizado de errores HTTP

n8n Workflow

Sub-flujo	Responsabilidad
FLUJO PRINCIPAL	Detección, priorización y persistencia de vulnerabilidades
FLUJO INYECTOR	Creación de activos de prueba para validar el pipeline
FLUJO ERRORES	Captura de fallos y notificación al equipo

Frontend (React)

Página	Responsabilidad
Dashboard	Métricas globales, MTTR, distribuciones, tendencias 30 días
Inventario	CRUD de activos, trigger de scan por activo
Vulnerabilidades	Listado filtrado, detalle, cambio de estado
Kanban	Gestión visual del ciclo de vida de vulnerabilidades

Modelo de datos de alto nivel

```
erDiagram
    environments {
        int id PK
        string name
        string business_criticality
        string description
        timestamp created_at
    }

    assets {
        int id PK
        string name
        string software
        string ecosystem
        string version
        int environment_id FK
        timestamp last_scan
        string description
    }

    vulnerabilities_audit {
        int id PK
        timestamp detected_at
        int asset_id FK
        string cve_id
        string ghsa_id
        varchar cvss
        boolean exploited
        string priority
        string decision
        string assigned_to
        string status
        timestamp resolved_at
        timestamp updated_at
    }

    users {
        int id PK
        string username
        string full_name
        string email
        string role
        timestamp created_at
    }

    system_errors {
```

```

    int id PK
    timestamp error_date
    string node_name
    string error_message
    string workflow_id
    string execution_id
}

vulnerabilities_ignored {
    int id PK
    string package_name
    string ecosystem
    string vulnerable_range
    string patched_version
    timestamp ignored_at
    int hit_count
}

cisa_kev_cache {
    int id PK
    timestamp fetched_at
    jsonb data
}

environments ||--o{ assets : "contiene"
assets ||--o{ vulnerabilities_audit : "tiene"
users ||--o{ vulnerabilities_audit : "asignado a"

```

Flujo de información — Scan por activo

```

sequenceDiagram
    participant U as Usuario
    participant FE as Frontend
    participant BE as Spring Boot
    participant N8N as n8n
    participant DB as PostgreSQL
    participant GHSA as GitHub Advisory
    participant CISA as CISA KEV

    U->>FE: Click "Disparar Scan" en activo X
    FE->>BE: POST /api/assets/{id}/scan
    BE->>DB: UPDATE assets SET last_scan = NOW()
    BE->>N8N: POST /webhook/auditoria-automatizada\n{activo_name: "react"}
    BE->>FE: 200 OK (activo actualizado)

    par Fetches paralelos
        N8N->>GHSA: GET /advisories?type=reviewed&per_page=100
        N8N->>CISA: GET /known_exploited_vulnerabilities.json
    end

    N8N->>DB: SELECT assets WHERE name = 'react'

```

```
N8N->>N8N: Risk Assessment Engine\n(CVSS + KEV + criticidad entorno)
N8N->>DB: INSERT vulnerabilities_audit\nON CONFLICT DO NOTHING
N8N->>DB: UPDATE assets SET last_scan
N8N->>Slack: Reporte técnico del activo
```

```
FE->>BE: GET /api/vulnerabilities
BE->>DB: SELECT * FROM vulnerabilities_audit\nWHERE asset_id = X
BE->>FE: Lista de vulnerabilidades detectadas
```

9. Riesgos del Proyecto

ID	Riesgo	Probabilidad	Impacto	Mitigación
R-01	El pipeline de scan no funciona el día de la demostración	Alta (actualmente roto)	Crítico	Prioridad máxima en Iter. 1. Demo en entorno controlado con datos pre-cargados como backup
R-02	CISA o GitHub API tienen downtime durante la demo	Media	Alto	Caché de resultados en DB (MEJ-03). Dataset de respaldo en fixtures
R-03	Credenciales de n8n no migran correctamente a nuevo servidor	Alta	Alto	Script setup-n8n.sh + documentación + prueba de instalación limpia antes de la demo
R-04	pgdata: external: true falla en instalación fresca	Alta (bug conocido)	Alto	Fix en Iter. 0 — cambiar a volumen gestionado
R-05	Duplicados de vulnerabilidades saturan la DB antes de la demo	Media	Alto	ON CONFLICT en Iter. 2 — fix de alta prioridad
R-06	n8n no levanta dentro del timeout de health check	Media	Alto	setup-n8n.sh con retry loop de 60 segundos
R-07	La demostración requiere auth y el sistema no la tiene	Media	Medio	Aclarar en el documento de tesis que auth es scope v2 — el MVP es un prototipo académico
R-08	MTTR no puede calcularse porque no hay vulns resueltas en el entorno de demo	Alta	Medio	Fixture de datos con vulns históricas en RESUELTA
R-09	Cambio de vulnPriority a inglés rompe estilos del Kanban	Media	Bajo	El componente VulnerabilityCard ya usa las claves en inglés — verificar antes de mergear
R-10	N8N_ENCRYPTION_KEY hardcodeada expuesta en repositorio público	Alta	Alto	Mover a .env antes de cualquier publicación del repo

10. Próximos Pasos

Acciones inmediatas (esta semana)

- ☐ [PB-047] Corregir `N8N_WEBHOOK_URL` en `docker-compose.yml` :
`http://n8n:5678/webhook/auditoria-automatizada`
- ☐ [PB-056] Corregir payload en `N8nWebhookService` : enviar `{activo_name: asset.getName()}`
- ☐ [PB-N8N-07] Agregar fallback `|| 'TODO5'` en nodo `Edit Fields` del workflow `n8n`
- ☐ [PB-057] Cambiar `pgdata: external: true` a volumen gestionado por Compose
- ☐ [PB-050] Crear `.env.example` con todas las variables documentadas
- ☐ [PB-N8N-04] Agregar `ON CONFLICT` en `Persist_Audit_Findings`
- ☐ [PB-063] Cambiar `filterDate default` en `Kanban.tsx` de hoy a vacío
- ☐ [PB-059] Corregir tipo `VulnPriority` en `frontend/src/types/index.ts`

Acciones de corto plazo (próximas 2 semanas)

- ☐ [PB-048, PB-049] Script `setup-n8n.sh` + habilitar `n8n Public API` + activar workflow
- ☐ [PB-N8N-02] Agregar `type=reviewed&per_page=100` a `Fetch_GitHub_Advisories`
- ☐ [PB-N8N-05, PB-N8N-06] Migrar queries SQL de `n8n` a modo parametrizado
- ☐ [PB-064, PB-065] Propagación de errores `n8n` al frontend
- ☐ [PB-002] Implementar MTTR: query SQL + DTO + card en Dashboard
- ☐ [PB-003, PB-066] Validación de transiciones de estado + handler 409
- ☐ [PB-051] Mover `N8N_ENCRYPTION_KEY` a variables de entorno

Acciones de mediano plazo (semanas 3-4)

- ☐ [PB-N8N-08] Tabla `cisa_kev_cache` + lógica de caché en workflow
- ☐ [PB-N8N-01] Paralelizar fetches CISA + GitHub
- ☐ [PB-060] Retry en nodos HTTP de `n8n`
- ☐ [PB-061, PB-062] Endpoints de scan por entorno y scan global
- ☐ [PB-N8N-06B, PB-N8N-06C] Fix Injector: `environment_id` fijo + scan automático post-inyección
- ☐ [PB-N8N-09] Fix URL Slack hardcodeada
- ☐ [PB-008, PB-009] Smoke tests + README de deployment
- ☐ Prueba de instalación limpia en máquina nueva antes de la demo

11. Conclusión

Estado real vs. estado percibido

[Análisis] El proyecto GEF Secure tiene una arquitectura conceptual sólida y bien diferenciada. El motor de riesgo contextualizado (CVSS + criticidad de entorno + CISA KEV + zero-day) es genuinamente valioso y representa una propuesta de valor real frente a herramientas de detección pura.

Sin embargo, existe una brecha significativa entre el potencial del producto y su estado operativo actual:

El feature más importante del producto — el pipeline de detección automática — está roto en 3 puntos simultáneos que se neutralizan mutuamente: el workflow `n8n` está inactivo, la URL del webhook es un placeholder, y el payload enviado desde `Spring Boot` no coincide con lo que `n8n` espera leer.

Qué eliminar

[Análisis] No hay sobreingeniería grave en el código actual. Las únicas áreas a simplificar son:

- El double-load en `AssetController` (dos queries al mismo activo)
- El `WebhookController` con endpoints muertos — documentar como dead code, no desarrollar

Qué agregar antes de la demo

[Análisis] Con orden de prioridad estricto:

1. Fix del pipeline completo (URL + payload + activación)
2. MTTR en el Dashboard (KPI central del PDF, ausente en código)
3. Deployment reproducible sin configuración manual

Evaluación del MVP

[Análisis] Corrigiendo los bugs críticos listados en las Iteraciones 0 y 1, el sistema tiene todas las condiciones para demostrar su propuesta de valor de manera convincente. El motor de riesgo ya está implementado y funciona correctamente — el problema no es la lógica, sino la plomería que la conecta.

El camino al MVP es de correcciones concretas, no de nuevas funcionalidades.

Deuda técnica a monitorear

[Análisis] Los tres ítems de mayor riesgo a largo plazo:

1. **Lógica de negocio en Code node de n8n** — no testeable, no versionable. Candidato a extracción post-MVP.
2. **CVSS como VARCHAR** — impide cualquier filtro o estadística numérica real. Requiere migración de schema.
3. **Ausencia de tests** — el sistema no tiene red de seguridad para refactors futuros.

Documento generado el 2026-07-29 mediante análisis profundo del código fuente, workflow n8n y documentación de proyecto. Fuentes analizadas: `Proyecto-tesis.pdf` , `GEF_Secure_Unificado.docx` , código fuente completo de backend, frontend y workflow n8n.